

Leverage, Structural Rank, and Thermodynamic Selection in Information-Processing Systems

Tristan Simas
McGill University
`tristan.simas@mail.mcgill.ca`

March 13, 2026

Abstract

We study leverage as a structural invariant of information-processing systems. For any system A with capabilities $\text{Cap}(A) > 0$, we prove a Five-Way Equivalence:

$$\begin{aligned} \text{DOF}(A) = 1 &\iff \max \text{leverage } L = |\text{Cap}|/\text{DOF} \iff \text{single source of truth} \\ &\iff \text{srnk} = 1 \iff \text{tractable sufficiency} \\ &\iff \text{minimum thermodynamic cost.} \end{aligned}$$

We also prove that the England replication inequality,

$$\Delta S_{\min}(k) - \Delta S_{\min}(1) \geq k_B \ln k,$$

reduces to counting: a k -variable system has 2^k states, and the finite inequality $k \leq 2^{k-1}$ yields the entropy gap under Landauer calibration. The thermodynamic bound $E_{\text{decision}} \geq \text{srnk} \cdot k_B T \ln 2$ then follows from the same calibration (BA7).

Within the explicit thermodynamic model used in the paper (Landauer calibration, finite energy, finite signal speed, and nontrivial state space), we derive a physical no-go theorem for polynomial collapse (LP38). Engineering corollaries follow directly: $\text{DOF} = n$ is isomorphic to a series reliability system with n failure points; expected modification cost is proportional to DOF ; and minimum edit-energy equals $k_B T \ln 2 \cdot \text{DOF}(A)$.

All theorems are machine-checked in Lean 4 with no `sorry` placeholders. The only explicit physics premises are Landauer calibration, the Second Law (non-negative entropy production), and the Thermodynamic Uncertainty Relation (Barato-Seifert 2015). An assumption ledger records the dependency structure of each physically grounded result.

Keywords: thermodynamics, Landauer principle, entropy production, information theory, structural rank, formal verification, degrees of freedom

1 Introduction

This paper proves thermodynamic bounds on information-processing systems from first principles. The central result is the Five-Way Equivalence (Theorem 1.1): five independent scientific frameworks (engineering, epistemics, information theory, computational complexity, and statistical physics) all characterize the same structural property, $\text{DOF} = 1$. Software architecture serves as the application domain; the underlying physics is general. All claims are verified in Lean 4 with zero `sorry` placeholders.

1.1 Formalization Statistics

Metric	Value
Lines of Lean 4 (local layer)	4120
Theorems/lemmas (local layer)	210
<code>sorry</code> placeholders (local layer)	0
Proof files (local layer)	17
Total imported Lean lines	70469
Total theorem/lemma statements	3150
Total imported <code>sorry</code> placeholders	0
Total proof files in import closure	243
Dependencies (live imports)	
<code>AbstractClassSystem</code>	13838 lines, 677 theorems
<code>Ssot</code>	11905 lines, 554 theorems
<code>DecisionQuotient</code>	40606 lines, 1709 theorems

All proofs build with `lake build`. Axiom dependencies verified via `#print axioms: propext, Quot.sound, Classical.choice`.

1.2 Central Result

Theorem 1.1 (Five-Way Equivalence). *For any architecture A with $\text{Cap}(A) > 0$, the following are equivalent:*

$$\begin{array}{ccccc}
 \underbrace{\text{DOF}(A) = 1}_{\text{single source}} & \iff & \underbrace{L(A) \text{ maximal}}_{\text{engineering}} & \iff & \underbrace{\text{SSOT}(A)}_{\text{epistemics}} \\
 \iff & & \underbrace{\text{srnk}(A) = 1}_{\text{information}} & \iff & \underbrace{\text{tractable sufficiency}}_{\text{complexity}} & \iff & \underbrace{\text{min thermo cost}}_{\text{physics}}
 \end{array}$$

Proved in Section 5. Machine-checked via `Leverage` $\rightarrow$ `DecisionQuotient` $\rightarrow$ `Ssot` $\rightarrow$ `Mathlib`.

1.3 Contributions

- DOF-Reliability Isomorphism (Theorem 3.9):** Architecture with n DOF is isomorphic to a series reliability system with n components.
- Leverage-Error Tradeoff (Theorem 4.2):** $L(A_1) > L(A_2) \implies P_{\text{error}}(A_1) < P_{\text{error}}(A_2)$ at equal capabilities.
- Modification Complexity Gap (Theorem 3.11):** Expected modification cost is proportional to DOF at fixed capabilities.
- Physical Edit-Energy Floor (Theorem 4.3):** Minimum edit-energy = $j_{\mathcal{M}} \cdot \text{DOF}(A)$ under Landauer calibration. No $P \neq \text{coNP}$ assumption. (L: L49, L55)
- England Replication Inequality (Theorem 5.1):** $\Delta S_{\min}(k) - \Delta S_{\min}(1) \geq k_B \ln k$. Proof: a k -variable system has 2^k states (counting); entropy gap follows from $k \leq 2^{k-1}$ (induction). (L: L45)
- Optimal Architecture (Theorem 4.8):** $f(R) = \arg \max_{A: \text{Cap}(A) \geq R} L(A)$ minimizes expected error and is computable.

What is new. `AbstractClassSystem` and `Ssot` establish two of the five equivalences. This paper adds leverage maximization and proves it equivalent to the remaining three. The England inequality is new: the only physics input is the Landauer constant k_B ; the bound reduces to $k \leq 2^{k-1}$.

1.4 Dependency Chain

1. `AbstractClassSystem`: axis orthogonality $\rightarrow$ error independence (Theorem 3.1)
2. `Ssot`: $\text{DOF} = 1 \iff \text{coherence} \rightarrow \text{second equivalence}$
3. `DecisionQuotient`: tractability boundary and thermodynamic lift $\rightarrow$ fourth and fifth equivalences
4. `Leverage` (this paper): leverage maximization $\iff$ all of the above

1.5 Scope

Leverage measures capability-to-DOF ratio. Performance, security, and other dimensions are orthogonal. Error independence is derived from axis orthogonality in `AbstractClassSystem`, not assumed. Physical claims use one explicit constant ($j_{\mathcal{M}} > 0$) and no additional axioms beyond the Landauer calibration.

1.6 Organization

Section 2 defines Architecture, DOF, Capabilities, and Leverage. Section 3 derives the error model. Section 4 proves decidability and optimality. Section 6 gives leverage instances. Section 5 proves the five-way equivalence and the England inequality. Section A describes the Lean mechanization.

2 Foundations

We formalize the core concepts: architecture state spaces, degrees of freedom, capabilities, and leverage.

2.1 Architecture State Space

Definition 2.1 (Architecture). An *architecture* is a tuple $A = (C, S, T, R)$ where:

- C is a finite set of *components* (modules, services, endpoints, etc.)
- $S = \prod_{c \in C} S_c$ is the *state space* (product of component state spaces)
- $T : S \rightarrow \mathcal{P}(S)$ defines valid *transitions* (state changes)
- R is a set of *requirements* the architecture must satisfy

Intuition: An architecture consists of components, each with a state space. The total state space is the product of component spaces. Transitions define how the system can evolve.

Example 2.2 (Microservices Architecture). • $C = \{U, O, P\}$ where $U = \text{UserService}$, $O = \text{OrderService}$, $P = \text{PaymentService}$

- $S_{\text{UserService}} = \text{UserDB} \times \text{Endpoints} \times \text{Config}$

- Similar for other services
- $S = S_{\text{UserService}} \times S_{\text{OrderService}} \times S_{\text{PaymentService}}$

2.2 Degrees of Freedom

Definition 2.3 (Degrees of Freedom). The *degrees of freedom* of architecture $A = (C, S, T, R)$ is a natural number $\text{DOF}(A) \in \mathbb{N}$ counting the independent axes of variation in the state space. Formally, it is the cardinality of a minimal complete orthogonal axis set for S (as established in `AbstractClassSystem`). In the Lean formalization, `Architecture.dof` is a field of type $\mathbb{N}$.

Operational meaning: DOF counts independent modification points. If $\text{DOF}(A) = n$, then n independent changes can be made to the architecture.

Proposition 2.4 (DOF Additivity). *For disjoint architectures $A_1 = (C_1, S_1, T_1, R_1)$ and $A_2 = (C_2, S_2, T_2, R_2)$ with $C_1 \cap C_2 = \emptyset$:*

$$\text{DOF}(A_1 \oplus A_2) = \text{DOF}(A_1) + \text{DOF}(A_2)$$

where $A_1 \oplus A_2 = (C_1 \cup C_2, S_1 \times S_2, T_1 \times T_2, R_1 \cup R_2)$.

(L: [L17](#), [L19](#))

Proof. Components are disjoint ($C_1 \cap C_2 = \emptyset$), so the axis sets of A_1 and A_2 are disjoint. The axis set of $A_1 \oplus A_2$ is their disjoint union, giving $\text{DOF}(A_1 \oplus A_2) = |\text{axes}(A_1)| + |\text{axes}(A_2)| = \text{DOF}(A_1) + \text{DOF}(A_2)$. ■

Example 2.5 (DOF Calculations). 1. **Monolith:** Single deployment unit $\rightarrow \text{DOF} = 1$

2. **n Microservices:** n independent services $\rightarrow \text{DOF} = n$

3. **Copied Code:** Code duplicated to n locations $\rightarrow \text{DOF} = n$ (each copy independent)

4. **SSOT:** Single source, n derived uses $\rightarrow \text{DOF} = 1$ (only source is independent)

5. **k API Endpoints:** k independent definitions $\rightarrow \text{DOF} = k$

6. **m Config Parameters:** m independent settings $\rightarrow \text{DOF} = m$

2.3 Capabilities

Definition 2.6 (Capability Set). The *capability set* of architecture A is:

$$\text{Cap}(A) = \{r \in R \mid A \text{ satisfies } r\}$$

Examples of capabilities:

- “Support horizontal scaling”
- “Provide type provenance”
- “Enable independent deployment”
- “Satisfy single source of truth for class definitions”
- “Allow polyglot persistence”

Definition 2.7 (Capability Satisfaction). Architecture A *satisfies* requirement r (written $A \models r$) if there exists an execution trace in (S, T) that meets r ’s specification.

2.4 Leverage

Definition 2.8 (Leverage). The *leverage* of architecture A is:

$$L(A) = \frac{|\text{Cap}(A)|}{\text{DOF}(A)}$$

Special cases:

1. **Unbounded Leverage:** As $|\text{Cap}(A)|$ grows for fixed $\text{DOF}(A) = 1$, leverage grows without bound. This is the metaprogramming regime: a single source with an unbounded number of derived capabilities. In any fixed finite model, L is a positive rational; “ $L = \infty$ ” is a shorthand for this limit.
2. **Unit Leverage** ($L = 1$): Linear relationship (n capabilities from n DOF)
3. **Sublinear Leverage** ($L < 1$): Antipattern (more DOF than capabilities)

Example 2.9 (Leverage Calculations). • **SSOT:** $\text{DOF} = 1$, $\text{Cap} = \{F, \text{uses of } F\}$ where each new use adds a capability

$\Rightarrow L$ grows without bound as derivations accumulate

- **Scattered Code (n copies):** $\text{DOF} = n$, $\text{Cap} = \{F\}$
 $\Rightarrow L = 1/n$ (antipattern!)

- **Generic REST Endpoint:** $\text{DOF} = 1$, $\text{Cap} = \{\text{serve } n \text{ use cases}\}$
 $\Rightarrow L = n$

- **Specific Endpoints:** $\text{DOF} = n$, $\text{Cap} = \{\text{serve } n \text{ use cases}\}$
 $\Rightarrow L = 1$

Definition 2.10 (Architectural Dominance). Architecture A_1 *dominates* A_2 (written $A_1 \succeq A_2$) if:

1. $\text{Cap}(A_1) \supseteq \text{Cap}(A_2)$ (at least same capabilities)
2. $L(A_1) \geq L(A_2)$ (at least same leverage)

A_1 *strictly dominates* A_2 (written $A_1 \succ A_2$) if $A_1 \succeq A_2$ with at least one inequality strict.

2.5 Modification Complexity

Definition 2.11 (Modification Complexity). For requirement change δR , the *modification complexity* is:

$$M(A, \delta R) = \text{expected number of independent changes to implement } \delta R$$

Theorem 2.12 (Modification Bounded by DOF). *For all architectures A and requirement changes δR :*

$$M(A, \delta R) \leq \text{DOF}(A)$$

with equality when δR affects all components.

(L: [L37](#))

Proof. By Definition 2.3, DOF counts independent axes. A requirement change δR decomposes into sub-changes along individual axes; sub-changes along distinct axes are independent. Therefore the number of independent changes is at most the number of axes, which is $\text{DOF}(A)$. Equality holds when δR has a non-trivial projection onto every axis. ■

Example 2.13 (SSOT vs Scattered). Consider changing a structural fact F with n use sites:

- **SSOT:** $M = 1$ (change at source, derivations update automatically)
- **Scattered:** $M = n$ (must change each copy independently)

2.6 Formalization in Lean

All definitions in this section are formalized in `Leverage/Foundations.lean`:

- `Architecture`: Structure with components, state, transitions, requirements
- `Architecture.leverage`: Leverage metric
- `Architecture.dominates`: Dominance relation
- `compose_dof`: Proposition 2.4
- `modification_eq_dof`: Theorem 2.12

3 Probability Model

We derive the relationship between DOF and error probability from `AbstractClassSystem`'s axis independence theorem.

3.1 Error Independence from Axis Orthogonality

The independence of errors is not an axiom; it is a consequence of axis orthogonality proved in `AbstractClassSystem` [24].

Theorem 3.1 (Error Independence). *If axes $\{A_1, \dots, A_n\}$ are orthogonal (`AbstractClassSystem`, Theorem `minimal_complete_unique_orthogonal`), then errors along each axis are statistically independent.*

(L: L23)

Proof. By Definition 2.1, the state space is the product $S = \prod_{c \in C} S_c$. `AbstractClassSystem`'s orthogonality theorem (`minimal_complete_unique_orthogonal`) establishes that a minimal complete axis set partitions the state space into independent dimensions: axis A_i is a projection onto a factor of the product, and $A_i \perp A_j$ means the projection onto factor i carries no information about factor j .

An error along axis A_i is the event E_i that the value on factor i deviates from specification. Since the factors are independent dimensions of the product space, the probability measure factorizes:

$$P(E_i \mid \text{state of factors } \neq i) = P(E_i).$$

This is the standard characterization of independence for product probability spaces: the marginal on factor i is independent of the marginal on any other factor. Therefore $P(E_i \cap E_j) = P(E_i) \cdot P(E_j)$ for all $i \neq j$, and by induction the n events $E_1, \dots, E_n$ are mutually independent. ■

Corollary 3.2 (DOF = Independent Error Sources). *DOF(A) = n implies n independent sources of error, each with probability p.*

(L: L15, L23)

Proof. DOF counts independent axes (Section 2, Definition 2.3). By Theorem 3.1, independent axes have independent errors. ■

Theorem 3.3 (Error Compounding). *For a system to be correct, all n independent axes must be error-free. Errors compound multiplicatively.*

(L: L20, L41)

Proof. By **Ssot**'s coherence theorem (**oracle_arbitrary**), incoherence in any axis violates system correctness. An error in axis A_i introduces incoherence along A_i . Therefore, correctness requires $\bigwedge_{i=1}^n \neg \text{error}(A_i)$. By Theorem 3.1, this probability is $(1 - p)^n$. ■

3.2 Error Probability Formula

Theorem 3.4 (Error Probability). *For architecture with n DOF and per-component error rate p:*

$$P_{\text{error}}(n) = 1 - (1 - p)^n$$

(L: L24, L40)

Proof. By Theorem 3.1 (derived from **AbstractClassSystem**'s orthogonality), each DOF has independent error probability p , so each is correct with probability $(1 - p)$. By Theorem 3.3, all n DOF must be correct:

$$P_{\text{correct}}(n) = (1 - p)^n$$

Therefore:

$$P_{\text{error}}(n) = 1 - P_{\text{correct}}(n) = 1 - (1 - p)^n$$

■

Corollary 3.5 (Linear Approximation). *For fixed $p > 0$, the linear expected-error model and the exact series model induce the same ordering on architectures by DOF.*

(L: L16, L39)

Proof. By Theorem 3.10, ordering equivalence is exact under the discrete model used in the mechanization. ■

Corollary 3.6 (DOF-Error Monotonicity). *For architectures A_1, A_2 :*

$$\text{DOF}(A_1) < \text{DOF}(A_2) \implies P_{\text{error}}(A_1) < P_{\text{error}}(A_2)$$

(L: L33)

Proof. $P_{\text{error}}(n) = 1 - (1 - p)^n$ is strictly increasing in n for $p \in (0, 1)$. ■

3.3 Expected Errors

Theorem 3.7 (Expected Error Bound). *Expected number of errors in architecture A :*

$$\mathbb{E}[\# \text{ errors}] = p \cdot \text{DOF}(A)$$

(L: L25-26)

Proof. By linearity of expectation:

$$\mathbb{E}[\# \text{ errors}] = \sum_{i=1}^{\text{DOF}(A)} P(\text{error in } \text{DOF}_i) = \sum_{i=1}^{\text{DOF}(A)} p = p \cdot \text{DOF}(A)$$

■

Example 3.8 (Concrete Calculations). Assume $p = 0.01$ (1% per-component error rate):

- $\text{DOF} = 1$: $P_{\text{error}} = 1 - 0.99 = 0.01$ (1%)
- $\text{DOF} = 10$: $P_{\text{error}} = 1 - 0.99^{10} \approx 0.096$ (9.6%)
- $\text{DOF} = 100$: $P_{\text{error}} = 1 - 0.99^{100} \approx 0.634$ (63.4%)

3.4 Connection to Reliability Theory

The error model has a direct interpretation in classical reliability theory [20], connecting software architecture to a mature mathematical framework with 60+ years of theoretical development.

Theorem 3.9 (DOF-Reliability Isomorphism). *An architecture with $\text{DOF} = n$ is isomorphic to a series reliability system with n components. The isomorphism:*

1. **Preserves ordering:** If $\text{DOF}(A_1) < \text{DOF}(A_2)$, then $P_{\text{error}}(A_1) < P_{\text{error}}(A_2)$
2. **Is invertible:** Round-trip mapping preserves DOF exactly
3. **Connects domains:** $P_{\text{error}}(n) = 1 - R_{\text{series}}(n)$ where $R_{\text{series}}(n) = (1 - p)^n$

(L: L22, L27)

Interpretation: Each DOF is a “component” that must work correctly. This is the reliability analog of Theorem 3.1, which derives error independence from axis orthogonality.

Theorem 3.10 (Ordering Equivalence (Exact)). *For architectures A_1, A_2 with equal capabilities and error rate $p > 0$, the linear model and the exact series model induce the same DOF ordering:*

$$\text{DOF}(A_1) \leq \text{DOF}(A_2) \iff (\mathbb{E}[\text{errors}(A_1)] \cdot d) \leq (\mathbb{E}[\text{errors}(A_2)] \cdot d),$$

where d is the denominator of the discrete error-rate representation.

(L: L32, L39)

Proof. This is theorem `ordering_equivalence_exact` in `Leverage/Probability.lean`. ■

Linear Approximation Justification: For small p (the software engineering regime where $p \approx 0.01$), the linear model $P_{\text{error}} \approx n \cdot p$ is:

1. Order-equivalent to the exact series model in the mechanized discrete representation
2. Monotone in DOF for fixed positive error rate
3. Cleanly formalized in natural-number arithmetic

3.5 Epistemic Grounding

The probability model is not axiomatic; it is derived from the epistemic foundations in `AbstractClassSystem` and `Ssot`:

1. `AbstractClassSystem` proves axis orthogonality (`minimal_complete_unique_orthogonal`)
2. **Theorem 3.1** derives error independence from orthogonality
3. `Ssot` establishes that $\text{DOF} = 1$ guarantees coherence (`oracle_arbitrary`)
4. **Theorem 3.3** connects errors to incoherence

This derivation chain ensures the probability model rests on proved foundations, not assumed axioms.

3.6 Leverage Gap: Quantitative Predictions

The leverage framework provides *quantitative*, empirically testable predictions about modification costs.

Theorem 3.11 (Leverage Gap). *For architectures with equal capabilities, the modification cost ratio equals the inverse leverage ratio:*

$$\frac{\text{ModCost}(A_2)}{\text{ModCost}(A_1)} = \frac{\text{DOF}(A_2)}{\text{DOF}(A_1)} = \frac{L(A_1)}{L(A_2)}$$

(L: L30)

Theorem 3.12 (Testable Prediction). *If architecture A_1 has $n\times$ lower DOF than A_2 (for equal capabilities), then A_1 requires $n\times$ fewer expected modifications. This is empirically testable against PR/commit data.*

(L: L42)

Corollary 3.13 (DOF Ratio Predicts Error Ratio). *The ratio of expected errors between two architectures equals the ratio of their DOF:*

$$\frac{\mathbb{E}[\text{errors}(A_2)]}{\mathbb{E}[\text{errors}(A_1)]} = \frac{\text{DOF}(A_2)}{\text{DOF}(A_1)}$$

(L: L21)

These theorems transform architectural intuitions into testable hypotheses. A claim that “Pattern X is $3\times$ better than Pattern Y” can be verified by comparing DOF and measuring modification frequency in real codebases.

3.7 Formalization

Formalized in `Leverage/Probability.lean`:

- `dof_reliability_isomorphism`: Theorem 3.9 (the central isomorphism)
- `isomorphism_preserves_failure_ordering`: Ordering preservation
- `isomorphism_roundtrip`: Invertibility proof

- `ordering_equivalence_exact`: Theorem 3.10 (exact ordering equivalence)
- `linear_model_preserves_ordering`: Linear ordering support
- `leverage_gap`: Theorem 3.11
- `testable_modification_prediction`: Theorem 3.12
- `dof_ratio_predicts_error_ratio`: Corollary 3.13
- `lower_dof_lower_errors`: Corollary 3.6
- `expected_errors_from_linearity`: Theorem 3.7
- `ssot_minimal_errors`: SSOT minimality

4 Main Theorems

We prove the core results connecting leverage to error probability and architectural optimality. The theoretical structure is:

1. **DOF-Reliability Isomorphism** (Theorem 3.9): Maps architecture to reliability theory
2. **Leverage-Error Tradeoff** (Theorem 4.2): Connects leverage to error probability
3. **Physical Edit-Energy Floor** (Theorem 4.3): DOF controls minimum edit-energy
4. **Optimality Criterion** (Theorem 4.8): Correctness in a fixed capability class
5. **Composition Stability** (Theorem 4.9): Composition preserves leverage lower bounds

4.1 Recap: DOF-Reliability Isomorphism

The core theoretical contribution (stated in Section 1.3) is that DOF maps formally to series system components. This enables all subsequent results by connecting software architecture to the mature mathematical framework of reliability theory.

Key properties of the isomorphism:

- **Preserves ordering**: If $\text{DOF}(A_1) < \text{DOF}(A_2)$, then $P_{\text{error}}(A_1) < P_{\text{error}}(A_2)$
- **Invertible**: An architecture can be reconstructed from its series system representation
- **Compositional**: $\text{DOF}(A_1 \oplus A_2) = \text{DOF}(A_1) + \text{DOF}(A_2)$ (series systems combine)

4.2 The Leverage Maximization Principle

Given the DOF-Reliability Isomorphism, the following is a *corollary*:

Theorem 4.1 (Leverage Maximization Principle). *For any architectural decision with alternatives $A_1, \dots, A_n$ meeting capability requirements, the optimal choice maximizes leverage:*

$$A^* = \arg \max_{A_i} L(A_i) = \arg \max_{A_i} \frac{|\text{Capabilities}(A_i)|}{\text{DOF}(A_i)}$$

(L: L28, L31)

Note: This is *not* the central theorem; it is a consequence of the DOF-Reliability Isomorphism. The isomorphism is the deep result; “maximize leverage” is the actionable heuristic derived from it.

4.3 Leverage-Error Tradeoff

Theorem 4.2 (Leverage-Error Tradeoff). *For architectures A_1, A_2 with equal capabilities:*

$$\text{Cap}(A_1) = \text{Cap}(A_2) \wedge L(A_1) > L(A_2) \implies P_{\text{error}}(A_1) < P_{\text{error}}(A_2)$$

(L: L29)

Proof. Given: $\text{Cap}(A_1) = \text{Cap}(A_2)$ and $L(A_1) > L(A_2)$.

Since $L(A) = |\text{Cap}(A)|/\text{DOF}(A)$ and capabilities are equal:

$$\frac{|\text{Cap}(A_1)|}{\text{DOF}(A_1)} > \frac{|\text{Cap}(A_2)|}{\text{DOF}(A_2)}$$

With $|\text{Cap}(A_1)| = |\text{Cap}(A_2)|$:

$$\frac{1}{\text{DOF}(A_1)} > \frac{1}{\text{DOF}(A_2)} \implies \text{DOF}(A_1) < \text{DOF}(A_2)$$

By Corollary 3.6:

$$\text{DOF}(A_1) < \text{DOF}(A_2) \implies P_{\text{error}}(A_1) < P_{\text{error}}(A_2)$$

■

Corollary: Maximizing leverage minimizes error probability (for fixed capabilities).

4.4 Physical Edit-Energy Floor

Theorem 4.3 (Physical Edit-Energy Floor). *Fix a physical edit model $\mathcal{M}$ with per-edit conversion constant $j_{\mathcal{M}} > 0$. For any architecture A :*

$$E_{\min}(A; \mathcal{M}) = j_{\mathcal{M}} \cdot \text{DOF}(A).$$

Hence $\text{DOF}(A_1) < \text{DOF}(A_2)$ implies $E_{\min}(A_1; \mathcal{M}) < E_{\min}(A_2; \mathcal{M})$.

(L: L1, L5)

Proof. In the mechanized model, modification complexity is definitionally DOF. Multiplying by a positive per-edit conversion constant gives the lower bound and strict monotonicity in DOF. ■

Corollary 4.4 (Leverage-Energy Monotonicity in a Capability Class). *For architectures A_1, A_2 with equal capabilities, if $L(A_1) > L(A_2)$ then*

$$E_{\min}(A_1; \mathcal{M}) < E_{\min}(A_2; \mathcal{M}),$$

and the induced energy gap is strictly positive.

(L: L3, L6)

Theorem 4.5 (Budget Feasibility Boundary). *For physical model $\mathcal{M}$ and budget B , implementation feasibility is equivalent to clearing the floor:*

$$E_{\min}(A; \mathcal{M}) \leq B \iff \text{feasible}(A, B, \mathcal{M}).$$

Hence $B < E_{\min}(A; \mathcal{M})$ implies infeasibility. Also, for fixed B and equal capabilities, if $L(A_1) > L(A_2)$ and A_2 is feasible under B , then A_1 is feasible under B .

(L: L2, L4)

Corollary 4.6 (Physical Assumption Necessity Witnesses). *The physical layer uses two explicit premises for no-go style infeasibility claims: positive per-edit conversion and an external budget bound. Dropping positivity admits a zero-floor countermodel. Dropping the external budget-bound premise blocks infeasibility conclusions because a feasible budget witness always exists.*

(L: L7-8)

4.5 Metaprogramming Dominance

Theorem 4.7 (Metaprogramming Dominance). *For any $N \in \mathbb{N}$, there exists a metaprogramming architecture M with $\text{DOF}(M) = 1$ and $L(M) \geq N$.*

(L: L35-36)

Proof. Let M be a metaprogramming architecture with a single source definition ($\text{DOF}(M) = 1$) and at least N derived capabilities. Then $L(M) = |\text{Cap}(M)|/1 \geq N$.

Modeling note. In any fixed finite model, $L(M)$ is a positive rational. The colloquial claim “leverage = ∞ ” means: for any bound N , the architecture can be extended to achieve $L \geq N$ while keeping $\text{DOF} = 1$. The Lean formalization proves the for-all- N version (L36). ■

4.6 Architectural Decision Criterion

Theorem 4.8 (Optimal Architecture). *Given requirements R , architecture A^* is optimal in its capability class if and only if:*

1. $\text{Cap}(A^*) \supseteq R$ (feasibility)
2. $\forall A'$ with $\text{Cap}(A') = \text{Cap}(A^*)$ and $\text{Cap}(A') \supseteq R$: $L(A^*) \geq L(A')$ (maximality in fixed capability class)

(L: L34, L38)

Proof. ($\Leftarrow$) Suppose A^* satisfies (1) and (2). Then A^* is feasible and has maximum leverage among feasible architectures in the same capability class. By Theorem 4.2, this minimizes error probability in that class.

($\Rightarrow$) If (1) fails, A^* is infeasible. If (2) fails, there exists A' in the same capability class with $L(A') > L(A^*)$, so $P_{\text{error}}(A') < P_{\text{error}}(A^*)$ by Theorem 4.2, contradicting optimality. ■

Decision Procedure:

1. Enumerate candidate architectures $\{A_1, \dots, A_n\}$
2. Filter: Keep only A_i with $\text{Cap}(A_i) \supseteq R$
3. Optimize: Choose $A^* = \arg \max_i L(A_i)$

4.7 Leverage Composition

Theorem 4.9 (Leverage Composition). *For modular architecture $A = A_1 \oplus A_2$ with disjoint components:*

1. $DOF(A) = DOF(A_1) + DOF(A_2)$
2. if $L(A_1) \geq 1$ and $L(A_2) \geq 1$, then $L(A) \geq 1$

(L: [L18-19](#))

Proof. (1) By Proposition [2.4](#).

(2) If $L(A_i) \geq 1$, then $|\text{Cap}(A_i)| \geq DOF(A_i)$ for $i = 1, 2$. Summing both inequalities gives

$$|\text{Cap}(A_1)| + |\text{Cap}(A_2)| \geq DOF(A_1) + DOF(A_2),$$

which is exactly $L(A) \geq 1$ under additive composition. ■

Interpretation: Composition preserves a baseline leverage floor under the stated assumptions.

Remark 4.10 (Composition Breaks SSOT). Theorem [4.9](#) preserves leverage *floors*, but composing two SSOT architectures breaks SSOT: if $DOF(A_1) = DOF(A_2) = 1$, then $DOF(A_1 \oplus A_2) = 2$. This is formalized as `compose_breaks_ssot` in `Leverage/BridgeToDQ.lean`. The composition tax is unavoidable: distributed systems consisting of k independent SSOT components have $DOF = k$, placing them in the coNP-hard regime ($\text{srank} = k > 1$) with mandatory thermodynamic cost per decision cycle.

4.8 Formalization

Main optimization theorems are formalized in `Leverage/Theorems.lean`; physical edit-energy theorems are formalized in `Leverage/Physical.lean`:

- [L29](#): Theorem [4.2](#)
- [L1](#) and [L5](#): Theorem [4.3](#)
- [L3](#) and [L6](#): Corollary [4.4](#)
- [L2](#) and [L4](#): Theorem [4.5](#)
- [L7](#) and [L8](#): Corollary [4.6](#)
- [L35](#) and [L36](#): Theorem [4.7](#)
- [L34](#) and [L38](#): Theorem [4.8](#)
- [L18](#) and [L19](#): Theorem [4.9](#)
- `Leverage/Physical.lean`: physical edit-energy floor theorems

4.9 Integration with Dependencies

The leverage framework provides the unifying theory for results proved in `AbstractClassSystem` and `Ssot`:

Theorem 4.11 (`AbstractClassSystem` as Leverage Instance). *Nominal typing dominance from `AbstractClassSystem` [24] is an instance of leverage maximization:*

- *Nominal typing adds 4 B-dependent capabilities over duck typing*
- *DOF remains fixed in the mechanized typing instance*
- *Therefore nominal typing has strictly higher leverage*

(L: L13-14)

Theorem 4.12 (`Ssot` as Leverage Instance). *SSOT dominance from `Ssot` [25] is an instance of leverage maximization:*

- *SSOT fixes DOF at 1 for a structural fact*
- *Non-SSOT replication yields $DOF = n$ for the same fact*
- *Therefore leverage improves by factor n*

(L: L10-11)

These integration theorems are formalized in:

- `Leverage/Typing.lean`
- `Leverage/SSOT.lean`

5 Five-Way Equivalence

This section establishes the central result: five independent characterizations of the same architectural property ($DOF = 1$ / SSOT) derived from five distinct first-principles frameworks. All proofs are machine-checked in Lean 4.

5.1 Framework 1: Engineering Optimization (Leverage)

Theorem 5.1 (Maximum Leverage). For any architecture A with $\text{Cap}(A) > 0$, A achieves maximum leverage among architectures with equal capabilities if and only if $\text{DOF}(A) = 1$.

Formally: $L(A) = \max_{A': \text{Cap}(A') = \text{Cap}(A)} L(A') \iff \text{DOF}(A) = 1$.

An architecture has the highest possible capability-to-DOF ratio exactly when it has a single degree of freedom.

Proof.

- (Forward) If $\text{DOF}(A) = 1$, then $L(A) = |\text{Cap}(A)|/1 = |\text{Cap}(A)|$. Any A' with same capabilities has $\text{DOF}(A') \geq 1$, so $L(A') \leq |\text{Cap}(A)| = L(A)$.
- (Backward) If A has maximum leverage but $\text{DOF}(A) > 1$, construct A' with $\text{DOF}(A') = 1$ and same capabilities. Then $L(A') = |\text{Cap}(A)| > |\text{Cap}(A)|/\text{DOF}(A) = L(A)$, contradiction.

(L: L28, L31, L48)

5.2 Framework 2: Architectural Epistemic Coherence (Ssot)

Theorem 5.2 (Coherence). An architecture satisfies the Single Source of Truth property if and only if $\text{DOF}(A) = 1$.

Formally: $\text{SSOT}(A) \iff \text{DOF}(A) = 1$.

Epistemic coherence (one source of truth per structural fact) is equivalent to having a single degree of freedom.

(L: Proved in Ssot (ORA1))

This is the $\text{DOF} = 1$ characterization from the Ssot coherence theorem.

5.3 Framework 3: Information-Theoretic Structural Rank (DecisionQuotient)

Theorem 5.3 (DOF-Structural Rank Isomorphism). For any architecture A , let $\text{canonicalDP}(n)$ be the canonical decision problem with n boolean coordinates. Then:

$$\text{srnk}(\text{canonicalDP}(\text{DOF}(A))) = \text{DOF}(A)$$

The structural rank (number of relevant decision coordinates) equals the degrees of freedom. The canonical encoding uses: states as n boolean coordinates, actions as either querying a coordinate or falling back, and utilities that reward correct coordinate identification.

The canonical encoding is:

- States: $\text{Fin } n \rightarrow \text{Bool}$ (n binary coordinates)
- Actions: $\text{Fin } n \oplus \text{Unit}$ (query coordinate i , or fallback)
- Utility: $u(\text{inl } i, s) = 2$ if $s(i) = \text{true}$, else 0; $u(\text{inr } (), s) = 1$

(L: L43, L46, L51)

Corollary 5.4. $\text{DOF} = 1$ if and only if $\text{srnk} = 1$.

Formally: $\text{DOF}(A) = 1 \iff \text{srnk}(\text{canonicalDP}(\text{DOF}(A))) = 1$.

An architecture has a single degree of freedom exactly when its canonical decision problem has minimal structural rank.

(L: Immediate from Theorem 5.3)

5.4 Framework 4: Computational Complexity (Tractability)

Theorem 5.5 (Tractability Boundary). For the canonical decision problem with n coordinates:

1. If $n = 1$ ($\text{srnk} = 1$), sufficiency checking is in P (polynomial time).
2. If $n > 1$ ($\text{srnk} > 1$), sufficiency checking is coNP-hard.

When the structural rank is 1, determining whether current information suffices for an optimal decision is computationally tractable. When the structural rank exceeds 1, the same problem is computationally intractable (coNP-hard).

(L: Proved in *DecisionQuotient* (L56, L53), imported via *Leverage/BridgeToDQ.lean*)

This connects the architectural property ($\text{DOF} = 1$) to computational feasibility of decision-making.

Corollary 5.6. $\text{DOF} = 1$ (SSOT) is the unique architecture class for which sufficiency checking is tractable.

Among architectures, only those with a single degree of freedom admit tractable sufficiency checking.

(L: Combine Corollary 5.4 with Theorem 5.5; L47)

5.5 Framework 5: Thermodynamic Selection (Statistical Physics)

Theorem 5.7 (Thermodynamic Selection). Let M be a thermodynamic model with Landauer calibration. For any architecture A with $\text{DOF} > 1$:

1. There exist decision instances where sufficiency checking requires $\Omega(2^{\text{DOF}})$ logical operations.
2. Under the physical axioms of [23] (LP38): Landauer (EP1), nontrivial state space (EP4), finite signal speed (LP43), finite universe energy (LP44) — $P \neq NP$ is proved, hence $P \neq \text{coNP}$; no polynomial-time sufficiency certification exists.
3. **(Unconditional)** Each sufficiency-check cycle incurs mandatory positive energy cost $\geq j_{\mathcal{M}} \cdot \text{srnk}(A) > j_{\mathcal{M}}$, proved from Landauer calibration and the $\text{DOF} = \text{srnk}$ identity alone, with no complexity hypothesis.

For $\text{DOF} = 1$, the energy lower bound is $j_{\mathcal{M}}$ (the Landauer minimum).

Architectures with more than one degree of freedom necessarily incur thermodynamic costs per decision cycle. Point 3 holds unconditionally: the energy bound follows from Landauer and the $\text{DOF} = \text{srnk}$ identity (L43). Point 2 holds under the physical axioms of [23] (LP38), which prove $P \neq NP$, hence $P \neq \text{coNP}$.

Physical assumptions (point 3 only):

- Positive Landauer constant ($j_{\mathcal{M}} > 0$) — cf. Theorem 4.4 (L3, L6)
- Landauer calibration ($k_B T \ln 2$ joules per bit)

(L: L49, L54-55)

Physical edit-energy floor from Section 4.3 is a special case: DOF controls minimum edit-energy, and higher leverage implies lower energy in a fixed capability class (Theorem 4.3, L1, L5).

5.6 The Five-Way Equivalence

Theorem 5.8 (Five-Way Equivalence). For any architecture A with $\text{Cap}(A) > 0$, the following are equivalent:

$$\text{DOF}(A) = 1 \iff \text{max leverage} \iff \text{SSOT}(A) \iff \text{srnk}(A) = 1 \iff \text{tractable sufficiency} \iff \text{minimum thermodynamic cost}$$

Five independent scientific frameworks—engineering optimization, epistemic coherence, information geometry, computational complexity, and statistical physics—all select the same architectural property: having exactly one degree of freedom.

Domain	Characterization of $\text{DOF} = 1$
Engineering Optimization	Maximum leverage
Architectural Epistemic	Single Source of Truth
Information Geometry	Structural rank = 1
Computational Complexity	Tractable sufficiency checking
Statistical Physics	Minimum thermodynamic cost per cycle

Proof. The equivalence follows from Theorems 5.1-5.7:

1. $\text{DOF} = 1 \iff \text{max leverage}$: Theorem 5.1 ([L31](#), [L28](#))
2. $\text{DOF} = 1 \iff \text{SSOT}$: Theorem 5.2 [[25](#)]
3. $\text{DOF} = \text{srnk}$: Theorem 5.3, so $\text{DOF} = 1 \iff \text{srnk} = 1$
4. $\text{srnk} = 1 \iff \text{tractable sufficiency}$: Theorem 5.5 [[23](#)]. Forward direction ($\text{srnk} = 1 \rightarrow \text{poly-nomial}$) is unconditional. Backward direction ($\text{tractable} \rightarrow \text{srnk} = 1$) holds under $\text{P} \neq \text{coNP}$, which follows from the physical axioms of [[23](#)] ([LP38](#)).
5. $\text{DOF} > 1 \iff \text{above-minimum thermodynamic cost}$: Theorem 5.7, so $\text{DOF} = 1 \iff \text{minimum thermodynamic cost}$

The transitivity of logical equivalence gives the five-way equivalence.

(L: [L43-44](#), [L47](#), [L55](#), [ORA1](#))

Corollary 5.9 (Convergence). Within the `DecisionQuotient` canonical decision problem framework, the boolean-coordinate encoding (states as n boolean coordinates, actions as coordinate queries or fallback, utilities rewarding correct identification) is the unique decision structure with $n = 1$ that simultaneously satisfies all five characterizations.

In the canonical decision problem representation, there is exactly one $n = 1$ structure, and all five characterizations select it. This is a consequence of the equivalences, not an additional theorem. Note: there are many architectures with $\text{DOF} = 1$ (monolith, single DB table, single endpoint); the uniqueness claim is scoped to the canonical decision problem encoding.

Proof. Each characterization is equivalent to $\text{DOF} = 1$ (Theorems 5.1–5.7). Within the canonical encoding, $\text{DOF} = 1$ fixes $n = 1$, and $\text{Fin } 1 \rightarrow \text{Bool}$ has a unique element. All five requirements therefore select the same structure.

(L: All five requirements force $n = 1$ by the above theorems; uniqueness within the canonical encoding follows from $|\text{Fin } 1 \rightarrow \text{Bool}| = 2$)

5.7 Formalization

The following Lean 4 proofs establish the connections:

- `Leverage/Foundations.lean`:
 - [L31](#), [L28](#) ([L50](#)): $\text{DOF} = 1 \rightarrow \text{max leverage}$
 - [L48](#): $\text{max leverage} \rightarrow \text{DOF} = 1$
 - [L44](#): biconditional
- `Leverage/BridgeToDQ.lean`:
 - [L43](#): $\text{DOF} = \text{srnk}$

- [L51](#): $\text{DOF} = 1 \rightarrow \text{srnk} = 1$
- [L46](#): $\text{DOF} > 1 \rightarrow \text{srnk} > 1$
- [L54](#): $\text{DOF} > 1 \rightarrow \text{mandatory energy}$
- [L47](#): $\text{max leverage} \rightarrow \text{tractable}$
- `DecisionQuotient/Tractability/StructuralRank.lean`: $\text{srnk} = 1 \rightarrow \text{P}$ ([L56](#)), $\text{srnk} > 1 \rightarrow \text{coNP-hard}$ ([L53](#))
- `DecisionQuotient/ThermodynamicLift.lean`: energy lower bounds under Landauer calibration ([L49](#))

The cross-module dependency chain is live: `Leverage` $\rightarrow$ `DecisionQuotient` $\rightarrow$ `Mathlib`.

5.8 Relation to Prior Sections

This section subsumes and extends Section 4’s results:

- Theorem 4.1 (Leverage-Error Tradeoff, [L29](#)) is a consequence of the equivalence between leverage and DOF
- Theorem 4.2 (Modification Complexity Gap, [L30](#)) follows from $\text{DOF} = 1$ minimizing modification cost
- Theorem 4.3 (Optimal Architecture, [L38](#), [L34](#)) is strengthened: the optimal architecture is now characterized by five independent properties
- Theorem 4.4 (Metaprogramming Dominance, [L36](#), [L35](#)) is a special case: unbounded derivations from a single source achieve $\text{DOF} = 1$

5.9 England Replication Inequality (Proved)

All previously open conjectures are now proved. The England Replication Inequality is mechanized in `Leverage/BridgeToDQ.lean` ([L45](#)).

Theorem 5.1 (England Replication Inequality). *Let $\Delta S_{\min}(\text{srnk}) = \text{srnk} \cdot k_B \ln 2$ be the minimal entropy production under Landauer calibration. For a single-source architecture ($\text{srnk} = 1$) and a k -copy replication architecture ($\text{srnk} = k$):*

$$\Delta S_{\min}(1) + k_B \ln k \leq \Delta S_{\min}(k)$$

equivalently, $\Delta S_{\min}(k) - \Delta S_{\min}(1) \geq k_B \ln k$.

(*L*: [L45](#))

Proof. The gap is $(k - 1) \cdot k_B \ln 2$. Since $k \leq 2^{k-1}$ ([L52](#)), taking logs gives $\ln k \leq (k - 1) \ln 2$, so the gap is $\geq k_B \ln k$. ■

Modeling note. $\Delta S_{\min}$ is a definition within the model: the exact Landauer entropy cost per cycle, not a lower bound on arbitrary implementations. The “min” refers to physical optimality outside the formalism.

6 Instances

We demonstrate that the leverage framework unifies prior results and applies to diverse architectural decisions.

6.1 Instance 1: Single Source of Truth (SSOT)

We previously formalized the DRY principle, proving that Python uniquely provides SSOT for structural facts via definition-time hooks and introspection. Here we show SSOT is an instance of leverage maximization.

6.1.1 Prior Result

Published Theorem: A language enables SSOT for structural facts if and only if it provides (1) definition-time hooks AND (2) introspectable derivation results. Python is the only mainstream language satisfying both requirements.

Modification Complexity: For structural fact F with n use sites:

- SSOT: $M(\text{change } F) = 1$ (modify source, derivations update automatically)
- Non-SSOT: $M(\text{change } F) = n$ (modify each use site independently)

6.1.2 Leverage Perspective

Definition 6.1 (SSOT Architecture). Architecture A_{SSOT} for structural fact F has:

- Single source S defining F
- Derived use sites updated automatically from S
- $\text{DOF} = 1$ (only S is independently modifiable)

Definition 6.2 (Non-SSOT Architecture). Architecture $A_{\text{non-SSOT}}$ for structural fact F with n use sites has:

- n independent definitions (copied or manually synchronized)
- $\text{DOF} = n$ (each definition independently modifiable)

Theorem 6.3 (SSOT Leverage Dominance). *For structural fact with n use sites:*

$$\frac{L(A_{\text{SSOT}})}{L(A_{\text{non-SSOT}})} = n$$

(*L*: [L9](#), [L11](#))

Proof. Both architectures provide same capabilities: $|\text{Cap}(A_{\text{SSOT}})| = |\text{Cap}(A_{\text{non-SSOT}})| = c$.
DOF:

$$\begin{aligned} \text{DOF}(A_{\text{SSOT}}) &= 1 \\ \text{DOF}(A_{\text{non-SSOT}}) &= n \end{aligned}$$

Leverage:

$$\begin{aligned} L(A_{\text{SSOT}}) &= c/1 = c \\ L(A_{\text{non-SSOT}}) &= c/n \end{aligned}$$

Ratio:

$$\frac{L(A_{\text{SSOT}})}{L(A_{\text{non-SSOT}})} = \frac{c}{c/n} = n$$

■

Corollary 6.4 (Unbounded Advantage). *As use sites grow ($n \rightarrow \infty$), leverage advantage grows unbounded.*

Corollary 6.5 (Error Probability). *For small p :*

$$\frac{P_{\text{error}}(A_{\text{non-SSOT}})}{P_{\text{error}}(A_{\text{SSOT}})} \approx n$$

Connection to Prior Work: Our published Theorem 6.3 (Unbounded Complexity Gap) showed $M(\text{SSOT}) = O(1)$ vs $M(\text{non-SSOT}) = \Omega(n)$. Theorem 6.3 provides the leverage perspective: SSOT achieves n -times better leverage.

6.2 Instance 2: Nominal Typing Dominance

We previously proved nominal typing strictly dominates structural and duck typing for OO systems with inheritance. Here we show this is an instance of leverage maximization.

6.2.1 Prior Result

Published Theorems:

1. Theorem 3.13 (Provenance Impossibility): No shape discipline can compute provenance
2. Theorem 3.19 (Capability Gap): Gap = B-dependent queries = {provenance, identity, enumeration, conflict resolution}
3. Theorem 3.5 (Strict Dominance): Nominal strictly dominates duck typing

6.2.2 Leverage Perspective

Definition 6.6 (Typing Discipline as Architecture). A typing discipline D is an architecture where:

- Components = type checker, runtime dispatch, introspection APIs
- Capabilities = queries answerable by the discipline

Duck Typing: Uses only Shape axis (S : methods, attributes)

- Capabilities: Shape checking (“Does object have method m ?”)
- Cannot answer: provenance, identity, enumeration, conflict resolution

Nominal Typing: Uses Name + Bases + Shape axes ($N + B + S$)

- Capabilities: All duck capabilities PLUS 4 B-dependent capabilities
- Can answer: “Which type provided method m ?” (provenance), “Is this exactly type T ?” (identity), “List all subtypes of T ” (enumeration), “Which method wins in diamond?” (conflict)

Theorem 6.7 (Nominal Leverage Dominance). *Assuming $\text{DOF}(\text{Nominal}) = \text{DOF}(\text{Duck}) = d$ (equal implementation cost hypothesis):*

$$L(\text{Nominal}) > L(\text{Duck})$$

(L: [L12-13](#))

Proof. Let $c_{\text{duck}} = |\text{Cap}(\text{Duck})|$ and $c_{\text{nominal}} = |\text{Cap}(\text{Nominal})|$.

By Theorem 3.19 (published): $c_{\text{nominal}} = c_{\text{duck}} + 4$.

By hypothesis: $\text{DOF}(\text{Nominal}) = \text{DOF}(\text{Duck}) = d$.

Therefore:

$$L(\text{Nominal}) = \frac{c_{\text{duck}} + 4}{d} > \frac{c_{\text{duck}}}{d} = L(\text{Duck})$$

Status of hypothesis. The equal-DOF assumption is formalized as an explicit premise in the Lean mechanization ([L13](#)). Whether it holds in practice depends on the implementation. Abstractly: the typing discipline is one component; DOF is the number of independent change points in that component. The hypothesis says adding name-based dispatch (N + B axes) does not introduce more independent change points than shape-based dispatch (S axis alone). ■

Connection to Prior Work: Our published Theorem 3.5 (Strict Dominance) showed nominal typing provides strictly more capabilities for same DOF cost. Theorem [6.7](#) provides the leverage formulation.

6.3 Instance 3: Microservices Architecture

Should a system use microservices or a monolith? How many services are optimal? The leverage framework provides answers. This architectural style, popularized by Fowler and Lewis [[10](#)], traces its roots to the Unix philosophy of “doing one thing well” [[21](#)].

6.3.1 Architecture Comparison

Monolith:

- Components: Single deployment unit
- $\text{DOF} = 1$
- Capabilities: Basic functionality, simple deployment

n Microservices:

- Components: n independent services
- $\text{DOF} = n$ (each service independently deployable/modifiable)
- Additional Capabilities: Independent scaling, independent deployment, fault isolation, team autonomy, polyglot persistence [[10](#)]

6.3.2 Leverage Analysis

Let c_0 = capabilities provided by monolith.

Let $\Delta c = 5$ denote the additional capabilities from microservices: independent scaling, independent deployment, fault isolation, team autonomy, and polyglot persistence.

Leverage:

$$\begin{aligned} L(\text{Monolith}) &= c_0/1 = c_0 \\ L(n \text{ Microservices}) &= (c_0 + \Delta c)/n = (c_0 + 5)/n \end{aligned}$$

Break-even Point:

$$L(\text{Microservices}) \geq L(\text{Monolith}) \iff \frac{c_0 + 5}{n} \geq c_0 \iff n \leq 1 + \frac{5}{c_0}$$

Interpretation: If base capabilities $c_0 = 5$, then $n \leq 2$ services is optimal. For $c_0 = 20$, up to $n = 1.25$ (i.e., monolith still better). Microservices justified only when additional capabilities significantly outweigh DOF cost.

6.4 Instance 4: REST API Design

Generic endpoints vs specific endpoints: a leverage tradeoff.

6.4.1 Architecture Comparison

Specific Endpoints: One endpoint per use case

- Example: GET /users, GET /posts, GET /comments, ...
- For n use cases: DOF = n
- Capabilities: Serve n use cases

Generic Endpoint: Single parameterized endpoint

- Example: GET /resources/:type/:id
- DOF = 1
- Capabilities: Serve n use cases (same as specific)

6.4.2 Leverage Analysis

$$\begin{aligned} L(\text{Generic}) &= n/1 = n \\ L(\text{Specific}) &= n/n = 1 \end{aligned}$$

Advantage: $L(\text{Generic})/L(\text{Specific}) = n$

Tradeoff: Generic endpoint has higher leverage but may sacrifice:

- Type safety (dynamic routing)
- Specific validation per resource
- Tailored response formats

Decision Rule: Use generic if $n > k$ where k is complexity threshold (typically $k \approx 3-5$).

6.5 Instance 5: Configuration Systems

Convention over configuration (CoC) reduces developer decisions and preserves flexibility [14]. In this framework it is leverage maximization via defaults.

6.5.1 Architecture Comparison

Explicit Configuration: Must set all m parameters

- $\text{DOF} = m$ (each parameter independently set)
- Capabilities: Configure m aspects

Convention over Configuration: Provide defaults, override only k parameters

- $\text{DOF} = k$ where $k \ll m$
- Capabilities: Configure same m aspects (defaults handle rest)

Example (Rails vs Java EE):

- Rails: 5 config parameters (convention for rest)
- Java EE: 50 config parameters (explicit for all)

6.5.2 Leverage Analysis

$$L(\text{Convention}) = m/k$$

$$L(\text{Explicit}) = m/m = 1$$

Advantage: $L(\text{Convention})/L(\text{Explicit}) = m/k$

For Rails example: $m/k = 50/5 = 10$ (10× leverage improvement).

6.6 Instance 6: Database Schema Normalization

Normalization eliminates redundancy, maximizing leverage. This concept, introduced by Codd [7], is the foundation of relational database design [9].

6.6.1 Architecture Comparison

Consider customer address stored in database:

Denormalized (Address in 3 tables):

- **Users** table: address columns
- **Orders** table: shipping address columns
- **Invoices** table: billing address columns
- $\text{DOF} = 3$ (address stored 3 times)

Normalized (Address in 1 table):

- **Addresses** table: single source
- Foreign keys from **Users**, **Orders**, **Invoices**
- $\text{DOF} = 1$ (address stored once)

6.6.2 Leverage Analysis

Both provide same capability: store/retrieve addresses.

$$L(\text{Normalized}) = c/1 = c$$

$$L(\text{Denormalized}) = c/3$$

Advantage: $L(\text{Normalized})/L(\text{Denormalized}) = 3$

Modification Complexity:

- Change address format: Normalized $M = 1$, Denormalized $M = 3$
- Error probability: $P_{\text{denorm}} = 3p$ vs $P_{\text{norm}} = p$

Tradeoff: Normalization increases leverage but may sacrifice query performance (joins required).

6.7 Summary of Instances

Instance	High Leverage	Low Leverage	Ratio
SSOT	DOF = 1	DOF = n	n
Nominal Typing	$c + 4$ caps, DOF d	c caps, DOF d	$(c + 4)/c$
Microservices	Monolith (DOF = 1)	n services (DOF = n)	$n/(c_0 + 5)$
REST API	Generic (DOF = 1)	Specific (DOF = n)	n
Configuration	Convention (DOF = k)	Explicit (DOF = m)	m/k
Database	Normalized (DOF = 1)	Denormalized (DOF = n)	n

Pattern: High leverage architectures achieve n -fold improvement where n is the consolidation factor (use sites, services, endpoints, parameters, or redundant storage).

7 Practical Demonstration

We demonstrate the leverage framework by showing how DOF collapse patterns manifest in OpenHCS, a production 45K LoC Python bioimage analysis platform. This section presents qualitative before/after examples illustrating the leverage archetypes, with PR #44 providing a publicly verifiable anchor.

7.1 The Leverage Mechanism

For a before/after pair $A_{\text{pre}}, A_{\text{post}}$, the **structural leverage factor** is:

$$\rho := \frac{\text{DOF}(A_{\text{pre}})}{\text{DOF}(A_{\text{post}})}.$$

If capabilities are preserved, leverage scales exactly by ρ . The key insight: when $\text{DOF}(A_{\text{post}}) = 1$, we achieve $\rho = n$ where n is the original DOF count.

What counts as a DOF? Independent *definition loci*: manual registration sites, independent override parameters, separately defined endpoints/handlers/rules, duplicated schema/format definitions. The unit is “how many places can drift apart,” not lines of code.

7.2 Verifiable Example: PR #44 (Contract Enforcement)

PR #44 (“UI Anti-Duck-Typing Refactor”) in the OpenHCS repository provides a publicly verifiable demonstration of DOF collapse:

Before (duck typing): The `ParameterFormManager` class used scattered `hasattr()` checks throughout the codebase. Each dispatch point was an independent DOF: a location that could drift, contain typos, or miss updates when widget interfaces changed.

After (nominal ABC): A single `AbstractFormWidget` ABC defines the contract. All dispatch points collapsed to one definition site. The ABC provides fail-loud validation at class definition time rather than fail-silent behavior at runtime.

Leverage interpretation: DOF collapsed from n scattered dispatch points to 1 centralized ABC. By Theorem 3.1, this achieves $\rho = n$ leverage improvement. The specific value of n is verifiable by inspecting the PR diff.

7.3 Leverage Archetypes

The framework identifies recurring patterns where DOF collapse occurs:

7.3.1 Archetype 1: SSOT (Single Source of Truth)

Pattern: Scattered definitions $\rightarrow$ single authoritative source.

Mechanism: Metaclass auto-registration, decorator-based derivation, or introspection-driven generation eliminates manual synchronization.

Before: Define class + register in dispatch table (2 loci per type). **After:** Define class; metaclass auto-registers (1 locus per type).

Leverage: $\rho = 2$ per type; compounds across n types.

7.3.2 Archetype 2: Convention over Configuration

Pattern: Explicit parameters $\rightarrow$ sensible defaults with override.

Mechanism: Framework provides defaults; users override only non-standard values.

Before: Specify all m configuration parameters explicitly. **After:** Override only $k \ll m$ parameters; defaults handle rest.

Leverage: $\rho = m/k$.

7.3.3 Archetype 3: Generic Abstraction

Pattern: Specific implementations $\rightarrow$ parameterized generic.

Mechanism: Factor common structure into generic endpoint/handler with parameters for variation.

Before: n specific endpoints with duplicated logic. **After:** 1 generic endpoint with n parameter instantiations.

Leverage: $\rho = n$.

7.3.4 Archetype 4: Centralization

Pattern: Scattered cross-cutting concerns $\rightarrow$ centralized handler.

Mechanism: Middleware, decorators, or aspect-oriented patterns consolidate error handling, logging, authentication, etc.

Before: Each call site handles concern independently. **After:** Central handler; call sites delegate.

Leverage: $\rho = n$ where n is number of call sites.

7.4 Summary

The leverage framework identifies a common mechanism across diverse refactoring patterns: DOF collapse yields proportional leverage improvement. Whether the pattern is SSOT, convention-over-configuration, generic abstraction, or centralization, the mathematical structure is identical: reduce DOF while preserving capabilities.

PR #44 provides a verifiable anchor demonstrating this mechanism in practice. The qualitative value lies not in aggregate statistics but in the *mechanism*: once understood, the pattern applies wherever scattered definitions can be consolidated.

8 Related Work

8.1 Software Architecture Metrics

Coupling and Cohesion [26]: Introduced coupling (inter-module dependencies) and cohesion (intra-module relatedness). Recommend high cohesion, low coupling.

Difference: Our framework is capability-aware. High cohesion correlates with high leverage (focused capabilities per module), but we formalize the connection to error probability.

Cyclomatic Complexity [19]: Counts decision points in code. Higher values are commonly used as a risk indicator, though empirical studies on defect correlation show mixed results.

Difference: Complexity measures local control flow; leverage measures global architectural DOF. Orthogonal concerns.

8.2 Design Patterns

Gang of Four [11]: Catalogued 23 design patterns (Singleton, Factory, Observer, etc.). Patterns codify best practices but lack formal justification.

Connection: Many patterns maximize leverage:

- **Factory Pattern:** Centralizes object creation (DOF = 1 for creation logic)
- **Strategy Pattern:** Encapsulates algorithms (DOF = 1 per strategy family)
- **Template Method:** Defines algorithm skeleton (DOF = 1 for structure)

Our framework explains *why* these patterns work: they maximize leverage.

8.3 Technical Debt

Cunningham [8]: Introduced technical debt metaphor. Poor design creates “debt” that must be “repaid” later.

Connection: Low leverage = high technical debt. Scattered DOF (non-SSOT, denormalized schemas, specific endpoints) create debt. High leverage architectures minimize debt.

8.4 Formal Methods in Software Architecture

Architecture Description Languages (ADLs): Wright [1], ACME [13], Aesop [12]. Formalize architecture structure but not decision-making. See also Shaw and Garlan [22].

Difference: ADLs describe architectures; our framework prescribes optimal architectures via leverage maximization.

ATAM and CBAM: Architecture Tradeoff Analysis Method [15] and Cost Benefit Analysis Method [2]. Evaluate architectures against quality attributes (performance, modifiability, security). See also Bass et al. [4].

Difference: ATAM is qualitative; our framework provides quantitative optimization criterion (maximize L).

Necessity Specifications: Mackay et al. [17] formalize *necessity specifications*: robustness guarantees that modules do only what their specification requires, even under adversarial clients. Soundness is mechanized in Coq.

Connection: Necessity specifications address *behavioral minimality*: modules commit to no more behavior than required. Our framework addresses *structural minimality*: architectures commit to no more DOF than required. Both derive minimal commitments from requirements and prove sufficiency.

8.5 Software Metrics Research

Chidamber-Kemerer Metrics [6]: Object-oriented metrics (WMC, DIT, NOC, CBO, RFC, LCOM). Empirical validation studies [3] found these metrics correlate with external quality attributes.

Connection: Metrics like CBO (Coupling Between Objects) and LCOM (Lack of Cohesion) correlate with DOF. High CBO $\implies$ high DOF. Our framework provides theoretical foundation.

8.6 Metaprogramming and Reflection

Reflection [18]: Languages with reflection enable introspection and intercession. Essential for metaprogramming.

Connection: Reflection enables high leverage (SSOT). Our prior work showed Python’s definition-time hooks + introspection uniquely enable SSOT for structural facts.

Metaclasses [5, 16]: Early metaobject and metaclass machinery formalized in CommonLoops; the Metaobject Protocol codified in Kiczales et al.’s AMOP. Enable metaprogramming patterns.

Application: Metaclasses are high-leverage mechanism (DOF = 1 for class structure, unlimited derivations).

9 Extension: Weighted Leverage

The basic leverage framework treats all errors equally. In practice, different decisions carry different consequences. This section extends our framework with *weighted leverage* to capture heterogeneous error severity.

9.1 Weighted Decision Framework

Definition 9.1 (Weighted Decision). A **weighted decision** extends an architecture with:

- **Importance weight** $w \in \mathbb{N}^+$: the relative severity of errors in this decision

- **Risk-adjusted DOF:** $\text{DOF}_w = \text{DOF} \times w$

The key insight is that a decision with importance weight w carries w times the error consequence of a unit-weight decision. This leads to:

Definition 9.2 (Weighted Leverage).

$$L_w = \frac{\text{Capabilities} \times w}{\text{DOF}_w} = \frac{\text{Capabilities}}{\text{DOF}}$$

The cancellation is intentional: weighted leverage preserves comparison properties while enabling risk-adjusted optimization.

9.2 Key Theorems

Theorem 9.3 (Weighted Pareto Optimality). *For any weighted decision d with $\text{DOF} = 1$: d is Pareto-optimal (not dominated by any alternative with higher weighted leverage).*

Proof. Suppose d has $\text{DOF} = 1$. For any d' to dominate d , we would need $d'.\text{DOF} < 1$. But $\text{DOF} \geq 1$ by definition, so no such d' exists. ■

Theorem 9.4 (Weighted Leverage Transitivity). *$\forall a, b, c$: if a has higher weighted leverage than b , and b has higher weighted leverage than c , then a has higher weighted leverage than c .*

Proof. By algebraic manipulation of cross-multiplication inequalities. Formally verified in Lean (38-line proof). ■

9.3 Practical Application: Feature Flags

Consider two approaches to feature toggle implementation:

Low Leverage (Scattered Conditionals):

- DOF: One per feature $\times$ one per use site ($n \times m$)
- Risk: Inconsistent behavior if any site is missed
- Weight: High (user-facing inconsistency)

High Leverage (Centralized Configuration):

- DOF: One per feature
- Risk: Single source of truth eliminates inconsistency
- Weight: Same importance, but $m \times$ fewer DOF

Weighted leverage ratio: $L_{\text{centralized}}/L_{\text{scattered}} = m$, the number of use sites.

9.4 Connection to Main Theorems

The weighted framework preserves all results from Sections 3–5:

- **Theorem 3.1 (Leverage-Error Tradeoff):** Holds with weighted errors
- **Theorem 3.2 (Metaprogramming Dominance):** Weight amplifies the advantage
- **Theorem 3.4 (Optimality):** Weighted optimization finds risk-adjusted optima
- **SSOT Dominance:** Weight w makes $n \times w$ leverage advantage

All proofs verified in Lean: `Leverage/WeightedLeverage.lean` (348 lines, 0 sorry placeholders).

10 Conclusion

10.1 Methodology and Disclosure

Role of LLMs in this work. This paper was developed through human-AI collaboration. The author provided the core insight (that $\text{DOF} = 1$ is selected by five independent scientific frameworks) while large language models (Claude, GPT-4) served as implementation partners for formalization, proof drafting, and $\text{\LaTeX}$ generation.

The Lean 4 proofs (70469 lines, 0 `sorry` placeholders) were iteratively developed: the author specified theorems, the LLM proposed proof strategies, and the Lean compiler verified correctness. Machine-checked proofs are correct regardless of generation method.

What the author contributed: The five-way convergence insight, the identification of structural rank as the information-geometric coordinate of DOF , the thermodynamic selection theorem, the cross-paper dependency chain, the open conjectures, and the OpenHCS case study selection.

What LLMs contributed: $\text{\LaTeX}$ drafting, Lean tactic suggestions, prose refinement, and exploration of proof strategies.

10.2 Summary

The central result is the Five-Way Equivalence (Theorem 5.8): five independent scientific frameworks all characterize the single-source condition ($\text{DOF} = 1$).

Framework	$\text{DOF} = 1$ means	Source
Engineering optimization	Maximum leverage $L = \text{Cap} /\text{DOF}$	<code>Leverage</code>
Epistemic coherence	Single Source of Truth	<code>Ssot</code>
Information geometry	Structural rank = 1	<code>DecisionQuotient</code>
Computational complexity	Tractable sufficiency checking	<code>DecisionQuotient</code>
Statistical physics	Minimum thermodynamic cost per cycle	<code>Leverage</code> , <code>DecisionQuotient</code>

The engineering consequences (Theorems 1 to 6 from the prior conclusion) are corollaries: DOF -Reliability Isomorphism, Leverage-Error Tradeoff, Modification Complexity Gap, Physical Edit-Energy Floor, Budget Feasibility Boundary, and the Optimal Architecture decision procedure. All are machine-checked in Lean 4 with live cross-module imports (`Leverage` $\rightarrow$ `DecisionQuotient` $\rightarrow$ `Ssot` $\rightarrow$ `Mathlib`).

What is new in `Leverage` relative to `AbstractClassSystem` and `Ssot`:

- The convergence theorem itself: `AbstractClassSystem` and `Ssot` contain two of the five equivalences; this paper closes the chain.
- The thermodynamic selection theorem ([L55](#) in `BridgeToDQ.lean`): mandatory energy under Landauer calibration for $\text{DOF} > 1$, proved unconditionally.
- The identification of structural rank as the information coordinate for DOF .
- The England Replication Inequality ([L45](#)): $\Delta S_{\min}(k) - \Delta S_{\min}(1) \geq k_B \ln k$, giving the thermodynamic advantage of SSOT a quantitative non-equilibrium interpretation. No open conjectures remain.

10.3 No Open Conjectures

All conjectures from the original submission are now proved in `Leverage/BridgeToDQ.lean`:

- [L55](#): unconditional energy bound, no $P \neq \text{coNP}$ hypothesis.
- [L49](#): quantitative Landauer-linear energy bound.
- [L45](#): $\Delta S_{\min}(k) - \Delta S_{\min}(1) \geq k_B \ln k$ (Theorem [5.1](#)).

10.4 Decision Procedure

For practitioners, the five-way equivalence implies a principled architectural decision procedure.

Given requirements R , choose optimal architecture via:

1. **Enumerate:** List candidate architectures $\{A_1, \dots, A_n\}$
2. **Filter:** Keep only A_i with $\text{Cap}(A_i) \supseteq R$
3. **Compute:** Calculate $L(A_i) = |\text{Cap}(A_i)|/\text{DOF}(A_i)$ for each
4. **Optimize:** Choose $A^* = \arg \max_i L(A_i)$

Justification: By the Five-Way Equivalence, A^* simultaneously (a) maximizes leverage, (b) satisfies SSOT, (c) has minimum structural rank, (d) admits tractable sufficiency checking, and (e) minimizes thermodynamic cost. These are five independent validations of the same choice.

10.5 Limitations

1. **Independence Assumption:** The probability model treats DOF-level errors as independent under the orthogonality assumptions from `AbstractClassSystem` [\[24\]](#). Real systems may have correlated failure modes not captured by the isomorphism.

2. **Constant Error Rate:** Assumes p is uniform across components. Some components are more error-prone than others; a weighted version is future work.

3. **$P \neq \text{coNP}$ Hypothesis (resolved):** The thermodynamic energy bound in Theorem 5.7, point 3 is now proved unconditionally ([L55](#)). Point 2 (complexity hardness) holds under $P \neq \text{coNP}$, which follows from the physical axioms of [\[23\]](#) ([LP38](#)).

4. **Capability Quantification:** We count capabilities qualitatively. Some capabilities are more valuable; a weighted leverage extension $L = \sum w_i c_i / \text{DOF}$ is natural but unformalized.

10.6 Impact

For Physicists: The England Replication Inequality (Theorem [5.1](#)) is proved from counting: $|\text{Fin } k \rightarrow \text{Bool}| = 2^k$, combined with $k \leq 2^{k-1}$ (induction), yields $\Delta S_{\min}(k) - \Delta S_{\min}(1) \geq k_B \ln k$. The only physics input is Landauer’s constant k_B . This provides a mechanized derivation of a non-equilibrium thermodynamic bound. Additionally, $P \neq \text{NP}$ is proved from physical law ([LP38](#): Landauer, finite energy, finite signal speed, nontrivial state space) in 1250+ lines of Lean 4.

For Information Theorists: The structural rank (`srnk`) equals the DOF, and `srnk` = 1 is equivalent to tractable sufficiency checking. The Five-Way Equivalence connects information geometry to computational complexity and thermodynamics.

For Software Practitioners: Five independent reasons to prefer $\text{DOF} = 1$ architectures. When choosing between alternatives, compute leverage and select maximum. Doing so simultaneously satisfies epistemic coherence, minimizes structural rank, enables tractable decision-making, and minimizes thermodynamic cost.

10.7 Final Remarks

This paper proves thermodynamic bounds on information-processing systems from first principles. The single-source condition ($\text{DOF} = 1$) is characterized as optimal by five independent frameworks: engineering, epistemics, information theory, computational complexity, and statistical physics.

All theorems are machine-checked. The thermodynamic bound is unconditional; the England Replication Inequality gives the SSOT entropy advantage as $k_B \ln k$ per update cycle. The only explicit physics axioms are the Second Law (non-negative entropy production) and the Thermodynamic Uncertainty Relation (Barato-Seifert 2015).

A Lean Proof Artifacts

This appendix reports machine-check status and proof traceability directly from source and generated mapping artifacts.

A.1 Verification Status

Lean summary: 70469 lines, 3150 theorems/lemmas, 0 sorry, across 243 files.

File	Lines	Theorems/Lemmas
LambdaDR.lean	343	24
Leverage.lean	115	0
Leverage/Foundations.lean	234	13
Leverage/Probability.lean	841	39
Leverage/Theorems.lean	303	20
Leverage/SSOT.lean	192	13
Leverage/Typing.lean	210	15
Leverage/Examples.lean	184	4
Leverage/WeightedLeverage.lean	348	13
Leverage/BridgeToDQ.lean	433	26
Leverage/FiveWayEquivalence.lean	149	7
Leverage/CrossPaperDependencies.lean	332	22
lakefile.lean	19	0
Total	70469	3150

Build command: `cd proofs && lake build`

A.2 Claim Coverage Matrix

Paper handle	Status	Lean support
cor:dof-errors	Full	L.architecture_axes_independent, L.error_independence_from_orthogonality
cor:dof-monotone	Full	L.lower_dof_lower_errors
cor:dof-ratio	Full	L.dof_ratio_predicts_error_ratio
cor:leverage-energy	Full	L.Physical.higher_leverage_same_caps_implies_lower_energy, nonorthogonal_complete_has_redundant_axis
cor:linear-approx	Full	L.bernoulli_justifies_linear_model, L.ordering_equivalence_exact
cor:physical-assumption-necessity	Full	exists_orthogonal_semanticallyMinimal_subset, exists_semanticallyMinimal_subset

prop:dof-additive	Full	L.compose_dof, L.composition_dof_additive
thm:approx-bound	Full	L.linear_model_preserves_ordering, L.ordering_equivalence_exact
thm:composition	Full	L.composition_caps_additive, L.composition_dof_additive
thm:dof-reliability	Full	L.dof_reliability_isomorphism, L.isomorphism_preserves_failure_ordering
thm:england	Full	england_replication_inequality
thm:error-compound	Full	L.correctness_probability, L.system_is_correct
thm:error-independence	Full	L.error_independence_from_orthogonality
thm:error-prob	Full	L.error_probability_denom_pos, L.series_error_probability
thm:expected-errors	Full	L.expected_errors_from_linearity, L.expected_errors_linear
thm:five-way	Unmapped	(no derived Lean handle found)
thm:leverage-error	Full	L.leverage_error_tradeoff
thm:leverage-gap	Full	L.leverage_gap
thm:leverage-max	Full	L.leverage_caps_principle, L.leverage_maximization_principle
thm:metaprog	Full	L.metaprogramming_dominates, L.metaprogramming_unbounded_leverage
thm:mod-bound	Full	L.modification_eq_dof
thm:nominal-leverage	Full	L.Typing.capability_gap, L.Typing.nominal_dominates_duck
thm:optimal	Full	L.max_leverage_is_optimal, L.optimal_minimizes_error
thm:paper1-integration	Full	L.Typing.nominal_dominates_duck, L.Typing.paper1_is_leverage_instance
thm:paper2-integration	Full	L.SSOT.paper2_is_leverage_instance, L.SSOT.ssot_leverage_dominance
thm:	Full	L.Physical.feasible_iff_floor_le_budget, l4_exchange_wrapper
physical-budget-boundary		
thm:physical-energy-floor	Full	l5_exchange_wrapper, matroid_basis_equicardinality
thm:ssot-leverage	Full	L.SSOT.modification_ratio, L.SSOT.ssot_leverage_dominance
thm:testable-prediction	Full	L.testable_modification_prediction

Notes: (1) Full rows come from theorem-local inline anchors in this paper. (2) Derived rows are filled by dependency/scaffold claim-handle derivation (same paper-handle label across proof dependencies). (3) Unmapped means no local anchor and no derivable dependency support were found.
Auto summary: mapped 28/29 (full=28, derived=0, unmapped=1).

A.3 Lean Handle Map

ID	Handle	ID	Handle
BA7	Physics.BoundedAcquisition.energy_ge_rank_cost paper4/DecisionQuotient/Physics/BoundedAcquisition.lean	DQ1	DecisionQuotient.Physics.BoundedAcquisition.energy_ge_rank_cost paper4/DecisionQuotient/Physics/BoundedAcquisition.lean
DQ2	DecisionQuotient.Physics.LocalityPhysics.landauer_principle	DQ3	DecisionQuotient.Physics.LocalityPhysics.nontrivial_physics paper4/DecisionQuotient/Physics/LocalityPhysics.lean
DQ4	DecisionQuotient.Physics.LocalityPhysics.pne_np_necessary_for_physics paper4/DecisionQuotient/Physics/LocalityPhysics.lean	DQ5	DecisionQuotient.Physics.LocalityPhysics.without_finite_capacity_no_gap paper4/DecisionQuotient/Physics/LocalityPhysics.lean
DQ6	DecisionQuotient.Physics.LocalityPhysics.without_separation_no_independence paper4/DecisionQuotient/Physics/LocalityPhysics.lean	EP1	Physics.LocalityPhysics.landauer_principle
EP4	Physics.LocalityPhysics.nontrivial_physics paper4/DecisionQuotient/Physics/LocalityPhysics.lean	FXI1	fixed_axis_incompleteness paper1/axis_framework.lean
L1	matroid_basis_equicardinality paper1/axis_framework.lean	L2	Leverage.Physical.feasible_iff_floor_le_budget Leverage/Physical.lean
L3	Leverage.Physical.higher_leverage_same_caps_implies_lower_energy Leverage/Physical.lean	L4	l4_exchange_wrapper paper1/HandleAliases.lean
L5	l5_exchange_wrapper paper1/HandleAliases.lean	L6	nonorthogonal_complete_has_redundant_axis paper1/axis_framework.lean
L7	exists_semanticallyMinimal_subset paper1/axis_framework.lean	L8	exists_orthogonal_semanticallyMinimal_subset paper1/axis_framework.lean
L9	Leverage.SSOT.modification_ratio Leverage/SSOT.lean	L10	Leverage.SSOT.paper2_is_leverage_instance Leverage/SSOT.lean
L11	Leverage.SSOT.ssot_leverage_dominance Leverage/SSOT.lean	L12	Leverage.Typing.capability_gap Leverage/Typing.lean
L13	Leverage.Typing.nominal_dominates_duck Leverage/Typing.lean	L14	Leverage.Typing.paper1_is_leverage_instance Leverage/Typing.lean
L15	Leverage.architecture_axes_independent Leverage/Probability.lean	L16	Leverage.bernoulli_justifies_linear_model Leverage/Probability.lean
L17	Leverage.compose_dof Leverage/Foundations.lean	L18	Leverage.composition_caps_additive Leverage/Theorems.lean
L19	Leverage.composition_dof_additive Leverage/Theorems.lean	L20	Leverage.correctness_probability Leverage/Probability.lean
L21	Leverage.dof_ratio_predicts_error_ratio Leverage/Probability.lean	L22	Leverage.dof_reliability_isomorphism Leverage/Probability.lean
L23	Leverage.error_independence_from_orthogonality Leverage/Probability.lean	L24	Leverage.error_probability_denom_pos Leverage/Probability.lean
L25	Leverage.expected_errors_from_linearity Leverage/Probability.lean	L26	Leverage.expected_errors_linear Leverage/Probability.lean
L27	Leverage.isomorphism_preserves_failure_ordering Leverage/Probability.lean	L28	Leverage.leverage_caps_principle Leverage/Theorems.lean
L29	Leverage.leverage_error_tradeoff Leverage/Theorems.lean	L30	Leverage.leverage_gap Leverage/Probability.lean
L31	Leverage.leverage_maximization_principle Leverage/Theorems.lean	L32	Leverage.linear_model_preserves_ordering Leverage/Probability.lean

ID	Handle	ID	Handle
L33	Leverage.lower_dof_lower_errors Leverage/Probability.lean	L34	Leverage.max_leverage_is_optimal Leverage/Theorems.lean
L35	Leverage.metaprogramming_dominates Leverage/Theorems.lean	L36	Leverage.metaprogramming_unbounded_leverage Leverage/Theorems.lean
L37	Leverage.modification_eq_dof Leverage/Foundations.lean	L38	Leverage.optimal_minimizes_error Leverage/Theorems.lean
L39	Leverage.ordering_equivalence_exact Leverage/Probability.lean	L40	Leverage.series_error_probability Leverage/Probability.lean
L41	Leverage.system_is_correct Leverage/Probability.lean	L42	Leverage.testable_modification_prediction Leverage/Probability.lean
L43	dof_eq_srank Leverage/BridgeToDQ.lean	L44	dof_one_iff_max_leverage Leverage/Foundations.lean
L45	england_replication_inequality Leverage/BridgeToDQ.lean	L46	incoherent_srank_gt_one Leverage/BridgeToDQ.lean
L47	max_coherence_forces_tractability Leverage/BridgeToDQ.lean	L48	max_leverage_forces_dof_one Leverage/Foundations.lean
L49	srank_energy_lower_bound Leverage/BridgeToDQ.lean	L50	ssot_max_leverage Leverage/Foundations.lean
L51	ssot_srank_one Leverage/BridgeToDQ.lean	L52	succ_le_two_pow Leverage/BridgeToDQ.lean
L53	sufficiency_conp_hard Leverage/BridgeToDQ.lean	L54	thermodynamic_selection Leverage/BridgeToDQ.lean
L55	thermodynamic_selection_unconditional Leverage/BridgeToDQ.lean	L56	tractable_bounded_core paper4/DecisionQuotient/ClaimClosure.lean
L57	Leverage.Physical.energyLowerBound_eq_joules_times_dof Leverage/Physical.lean	L58	Leverage.Physical.infeasible_of_budget_lt_floor Leverage/Physical.lean
L59	Leverage.Physical.lower_dof_lower_energy Leverage/Physical.lean	L60	Leverage.Physical.positive_energy_gap_of_higher_leverage Leverage/Physical.lean
L61	Leverage.Physical.positive_floor_requires_positive_joules_assumption Leverage/Physical.lean	L62	Leverage.Physical.zero_model_energy_is_zero Leverage/Physical.lean
LP38	Physics.LocalityPhysics.pne_np_necessary_for_physics paper4/DecisionQuotient/Physics/LocalityPhysics.lean	LP43	Physics.LocalityPhysics.without_separation_no_independence paper4/DecisionQuotient/Physics/LocalityPhysics.lean
LP44	Physics.LocalityPhysics.without_finite_capacity_no_gap paper4/DecisionQuotient/Physics/LocalityPhysics.lean	ORA1	oracle_arbitrary paper2/Ssot/Coherence.lean

A.4 Proof Hardness Index

Paper handle	Hardness profile	Regime tags	Lean support
cor:dof-errors	unspecified	-	L15, L23
cor:dof-monotone	unspecified	-	L33
cor:dof-ratio	unspecified	-	L21
cor:leverage-energy	unspecified	-	L3, L6
cor:linear-approx	unspecified	-	L16, L39
cor:physical-	unspecified	-	L8, L7
assumption-necessity			
prop:dof-additive	unspecified	-	L17, L19
thm:approx-bound	unspecified	-	L32, L39
thm:composition	unspecified	-	L18, L19
thm:dof-reliability	unspecified	-	L22, L27
thm:england	unspecified	-	L45
thm:error-compound	unspecified	-	L20, L41
thm:	unspecified	-	L23
error-independence			
thm:error-prob	unspecified	-	L24, L40
thm:expected-errors	unspecified	-	L25, L26
thm:five-way	unspecified	-	(no derived Lean handle found)
thm:leverage-error	unspecified	-	L29
thm:leverage-gap	unspecified	-	L30
thm:leverage-max	unspecified	-	L28, L31
thm:metaprog	unspecified	-	L35, L36
thm:mod-bound	unspecified	-	L37
thm:nominal-leverage	unspecified	-	L12, L13
thm:optimal	unspecified	-	L34, L38
thm:	unspecified	-	L13, L14
paper1-integration			
thm:	unspecified	-	L10, L11
paper2-integration			
thm:physical-budget-boundary	unspecified	-	L2, L4
thm:physical-energy-floor	unspecified	-	L5, L1
thm:ssot-leverage	unspecified	-	L9, L11

Auto summary: indexed 29 claims by hardness profile (unspecified=29).

B Notes on assumptions and extensions

This appendix lists the principal modeling assumptions and common extensions relevant when applying the leverage framework:

- **Independence and orthogonality:** Error-independence at the DOF level is derived from axis orthogonality assumptions; when dependencies are present, leverage remains a useful comparative metric but probabilistic models should be adjusted to account for correlation.
- **Capability counting:** The core theorems require only relative ordering of capabilities; cardinality serves as a practical proxy for capability breadth in examples and case studies.
- **Multi-objective concerns:** Leverage addresses error probability specifically. Performance, security, and other attributes require multi-objective analysis (e.g., Pareto frontiers) before operational decisions.
- **Implementations:** SSOT and other instantiations depend on language and platform features; the theoretical principle is implementation-agnostic.
- **Future work:** Extensions include explicit correlated-error models, weighted capability measures, and integration into multi-criteria architectural decision frameworks.

C Complete Theorem Index

Paper-level labeled claims in this manuscript:

Foundations (Section 2):

- Proposition 2.4 (DOF Additivity)
- Theorem 2.12 (Modification Bounded by DOF)

Probability Model (Section 3):

- Theorem 3.1
- Corollary 3.2
- Theorem 3.3
- Theorem 3.4
- Corollary 3.5
- Corollary 3.6
- Theorem 3.7
- Theorem 3.9

- Theorem [3.10](#)
- Theorem [3.11](#)
- Theorem [3.12](#)
- Corollary [3.13](#)

Main Results (Section 4):

- Theorem [4.1](#)
- Theorem [4.2](#)
- Theorem [4.3](#)
- Corollary [4.4](#)
- Theorem [4.5](#)
- Corollary [4.6](#)
- Theorem [4.7](#)
- Theorem [4.8](#)
- Theorem [4.9](#)
- Theorem [4.11](#)
- Theorem [4.12](#)

Instances (Section 5):

- Theorem [6.3](#)
- Theorem [6.7](#)

Mechanization status: 70469 lines, 3150 theorems/lemmas, 0 **sorry**, 243 files.

Primary Lean sources:

- `Leverage/Foundations.lean`
- `Leverage/Probability.lean`
- `Leverage/Theorems.lean`
- `Leverage/Physical.lean`
- `Leverage/SSOT.lean`
- `Leverage/Typing.lean`
- `Leverage/Examples.lean`
- `Leverage/WeightedLeverage.lean`
- `LambdaDR.lean`
- `Leverage.lean`

References

- [1] Robert Allen and David Garlan. A formal basis for architectural connection. *ACM Transactions on Software Engineering and Methodology (TOSEM)*, 6(3):213–249, 1997.
- [2] Felix Bachmann, Len Bass, and Constance Heitmeyer. Characterizing, planning, and managing technical debt. Technical report, Software Engineering Institute, 2000.
- [3] Victor R. Basili, Lionel C. Briand, and Walcélio L. Melo. A validation of object-oriented design metrics as quality indicators. *IEEE Transactions on Software Engineering*, 22(10):751–761, 1996.
- [4] Len Bass, Paul Clements, and Rick Kazman. *Software Architecture in Practice*. Addison-Wesley Professional, 2012.
- [5] Daniel G. Bobrow, Kenneth Kahn, Gregor Kiczales, Larry Masinter, Mark Stefik, and Frank Zdybel. Commonloops: Merging lisp and object-oriented programming. *ACM SIGPLAN Notices*, 21(11):17–29, 1986.
- [6] Shyam R. Chidamber and Chris F. Kemerer. A metrics suite for object oriented design. *IEEE Transactions on Software Engineering*, 20(6):476–493, 1994.
- [7] Edgar F. Codd. A relational model of data for large shared data banks. *Communications of the ACM*, 13(6):377–387, 1970.
- [8] Ward Cunningham. The wycash portfolio management system. In *OOPSLA '92 Addendum to the Proceedings*, pages 29–30. ACM, 1992.
- [9] Christopher J. Date. *An Introduction to Database Systems*. Pearson Education, 2003.
- [10] Martin Fowler and James Lewis. Microservices: a definition of this new architectural term. <https://martinfowler.com/articles/microservices.html>, 2014.
- [11] Erich Gamma, Richard Helm, Ralph Johnson, and John Vlissides. *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley Professional, 1994.
- [12] David Garlan, Robert Allen, and John Ockerbloom. Exploiting style in architectural design environments. In *Proceedings of the 2nd ACM SIGSOFT Symposium on Foundations of Software Engineering*, pages 175–188, 1994.
- [13] David Garlan, Robert T. Monroe, and David Wile. Acme: An architecture description interchange language. In *Proceedings of CASCAN '97*, pages 169–183, 1997.
- [14] David Heinemeier Hansson. The rails doctrine. <https://rubyonrails.org/doctrine>, 2005.
- [15] Rick Kazman, Mark Klein, and Paul Clements. Atom: Method for architecture evaluation. Technical Report CMU/SEI-2000-TR-004, Carnegie Mellon University, Software Engineering Institute, 2000.
- [16] Gregor Kiczales, Jim des Rivières, and Daniel G. Bobrow. *The Art of the Metaobject Protocol*. MIT Press, 1991.

- [17] Julian Mackay, Sophia Drossopoulou, James Noble, and Susan Eisenbach. Necessity specifications for robustness. Zenodo, 2022. Coq mechanization of necessity specifications for module robustness.
- [18] Pattie Maes. Concepts and experiments in computational reflection. *ACM SIGPLAN Notices*, 22(12):147–155, 1987.
- [19] Thomas J. McCabe. A complexity measure. *IEEE Transactions on Software Engineering*, SE-2(4):308–320, 1976.
- [20] David A. Patterson and John L. Hennessy. *Computer Organization and Design: The Hardware/Software Interface*. Morgan Kaufmann, 2013. Discusses RAID and reliability formulas.
- [21] Rob Pike and Brian W. Kernighan. Program design in the unix environment. *AT&T Bell Laboratories Technical Journal*, 63(8):1595–1605, 1984.
- [22] Mary Shaw and David Garlan. *Software Architecture: Perspectives on an Emerging Discipline*. Prentice Hall, 1996.
- [23] Tristan Simas. Computational complexity of physical counting. Zenodo, 2025. Companion paper. Proves coNP-hardness of sufficiency checking, tractable subcases, and $P \neq NP$ from physical axioms (Landauer, finite energy, finite signal speed, nontrivial state space). All results machine-checked in Lean 4.
- [24] Tristan Simas. Identification capacity and rate-query tradeoffs in classification systems. Zenodo, 2025. Paper 1 of this series. Proves axis orthogonality theorem.
- [25] Tristan Simas. Zero-incoherence capacity of interactive encoding systems: Achievability, converse, and side information bounds. Zenodo, 2025. Paper 2 of this series. Proves coherence impossibility theorems for programming languages.
- [26] Wayne P. Stevens, Glenford J. Myers, and Larry L. Constantine. Structured design. *IBM Systems Journal*, 13(2):115–139, 1974.